

Azure Service Bus – Receiving Messages from a Topic Subscription

1. Overview

In Azure Service Bus, when a message is published to a **Topic**, it is delivered to the appropriate **Subscriptions**.

To receive a message from a topic, we don't receive the message directly from the topic.

Instead, we receive the message from a **Subscription**.

Basic Flow

```
Publisher
  ↓
  Topic
  ↓
Subscription S1
  ↓
  Receiver
```

For example:

```
Topic: FirstTopic
  |
  ├── S1
  |
  └── S2
```

If a message is available in S1, an application can receive it from **S1**.

2. Topic vs Subscription Receiver

This is an important difference to remember.

Queue Receiver

When receiving from a queue, we only need the:

Queue Name

Example:

```
ServiceBusReceiver receiver =  
    client.CreateReceiver(queueName);
```

Topic Subscription Receiver

When receiving from a topic subscription, we need **two names**:

1. Topic name
2. Subscription name

Example:

```
ServiceBusReceiver receiver =  
    client.CreateReceiver(topicName, subscriptionName);
```

Easy Way to Remember

Queue:
Receiver → Queue Name

Topic:
Receiver → Topic Name + Subscription Name

3. Example

Suppose we have:

```
Topic: FirstTopic  
  
Subscriptions:  
    S1  
    S2
```

And S1 contains:

```
Hello Topic from Visual Studio
```

We want to read this message from S1.

The flow is:

```
FirstTopic
  |
  └─ S1
      |
      └─ "Hello Topic from Visual Studio"
          ↓
        Receiver
```

4. Required NuGet Package

For .NET applications, install:

```
Azure.Messaging.ServiceBus
```

This package provides classes such as:

```
ServiceBusClient
ServiceBusReceiver
ServiceBusMessage
ServiceBusSender
```

5. Connection String

We need a connection string to connect to the Azure Service Bus namespace.

Example:

```
string connectionString = "<connection-string>";
```

The connection string is used to create the `ServiceBusClient`.

Security Note

Do not expose a real connection string in source code or commit it to Git.

For production applications, use secure options such as:

- Azure Key Vault
- Managed Identity
- Environment variables
- Secure application configuration

6. Create ServiceBusClient

Create a `ServiceBusClient` using the connection string.

```
ServiceBusClient client =  
    new ServiceBusClient(connectionString);
```

The `ServiceBusClient` is used to communicate with Azure Service Bus.

7. Create a Receiver for a Topic Subscription

Suppose:

```
Topic Name = FirstTopic  
Subscription Name = S1
```

Create the receiver:

```
ServiceBusReceiver receiver =  
    client.CreateReceiver(  
        "FirstTopic",  
        "S1");
```

Notice that we provide **both**:

```
Topic Name
Subscription Name
```

8. Why Do We Need the Subscription Name?

A topic can have multiple subscriptions.

For example:

```
      FirstTopic
      /
     ↓      ↓
    S1     S2
```

Suppose the same message is available in both:

```
S1 → Message A
S2 → Message A
```

If the application wants to receive from S1, it must specify:

```
Topic = FirstTopic
Subscription = S1
```

If it wants to receive from S2:

```
Topic = FirstTopic
Subscription = S2
```

Therefore, the subscription name tells Service Bus **which subscription the receiver should read from**.

9. Receive the Message

Once the receiver is created, use:

```
ServiceBusReceivedMessage message =  
    await receiver.ReceiveMessageAsync();
```

This reads a message from the specified subscription.

Example:

```
var message =  
    await receiver.ReceiveMessageAsync();  
  
Console.WriteLine(message.Body);
```

The output could be:

```
Hello Topic from Visual Studio
```

10. Complete Example

A simple topic subscription receiver looks like this:

```
using Azure.Messaging.ServiceBus;  
  
string connectionString = "<connection-string>";  
  
string topicName = "FirstTopic";  
string subscriptionName = "S1";  
  
ServiceBusClient client =  
    new ServiceBusClient(connectionString);  
  
ServiceBusReceiver receiver =  
    client.CreateReceiver(  
        topicName,  
        subscriptionName);  
  
ServiceBusReceivedMessage message =  
    await receiver.ReceiveMessageAsync();  
  
Console.WriteLine(message.Body);
```

11. Message Flow in the Example

The application executes:

```
Step 1
Create ServiceBusClient
    ↓
Step 2
Create receiver
    ↓
Topic = FirstTopic
Subscription = S1
    ↓
Step 3
ReceiveMessageAsync()
    ↓
Step 4
Message received from S1
```

The complete flow:

```
Azure Service Bus

FirstTopic
|
├── S1
|   |
|   └── "Hello Topic from Visual Studio"
|       ↓
|       Receiver
|
└── S2
```

The receiver is specifically connected to **S1**, so it reads the message from S1.

12. Peek-Lock and Completing the Message

By default, the receiver uses **Peek-Lock** mode.

That means receiving the message does not permanently delete it immediately.

After successfully processing the message, we should complete it:

```
await receiver.CompleteMessageAsync(message);
```

The flow becomes:

```
Subscription
  ↓
Receive
  ↓
Message Locked
  ↓
Process Message
  ↓
CompleteMessageAsync()
  ↓
Message Removed
```

If you use Receive-and-Delete mode, the message is deleted as soon as it is received.

13. Example with Message Completion

```
ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();

if (message != null)
{
    Console.WriteLine(message.Body);

    await receiver.CompleteMessageAsync(message);
}
```

This is useful when using Peek-Lock mode and you want to remove the message only after successful processing.

14. Example from the Video

The Azure Portal contained:

```
Topic: FirstTopic
```

```
Subscriptions:
```

```
S1
```

```
S2
```

The S1 subscription contained:

```
1 message
```

The message body was:

```
Hello topic from C# Visual Studio
```

The application created a receiver using:

```
ServiceBusReceiver receiver =  
    client.CreateReceiver(  
        topicName,  
        subscriptionName);
```

Where:

```
topicName = FirstTopic  
subscriptionName = S1
```

Then it received the message:

```
var message =  
    await receiver.ReceiveMessageAsync();
```

The received message body was:

```
Hello topic from C# Visual Studio
```

This matched the message present in the S1 subscription in the Azure Portal.

15. Queue Receiver vs Topic Subscription Receiver

This is a very important interview question.

Feature	Queue	Topic Subscription
Receiver destination	Queue	Subscription
Required names	Queue name	Topic name + Subscription name
Example	<code>CreateReceiver("Q1")</code>	<code>CreateReceiver("Topic1", "S1")</code>
Messaging pattern	Point-to-point	Publish-subscribe

Queue

```
var receiver =  
    client.CreateReceiver("Q1");
```

Topic Subscription

```
var receiver =  
    client.CreateReceiver(  
        "Topic1",  
        "S1");
```

16. Real-Time Example

Suppose an application publishes an order event to:

```
Orders Topic
```

The topic contains:

```
Orders Topic
├── Payment
├── Inventory
└── Notification
```

A payment application should receive messages from:

```
Orders Topic
    ↓
Payment Subscription
```

An inventory application should receive from:

```
Orders Topic
    ↓
Inventory Subscription
```

Each application creates its receiver using the topic and its subscription:

```
client.CreateReceiver(
    "Orders",
    "Payment");
```

or:

```
client.CreateReceiver(
    "Orders",
    "Inventory");
```

17. Important Interview Questions

Q1. Can we receive a message directly from a Topic?

Answer:

No. Messages published to a topic are received through its **subscriptions**.

Publisher → Topic → Subscription → Receiver

Q2. What information is required to create a receiver for a Topic?

Answer:

We need:

1. Topic name
2. Subscription name

Example:

```
client.CreateReceiver(  
    topicName,  
    subscriptionName);
```

Q3. What is the difference between creating a Queue receiver and a Topic receiver?

Answer:

For a queue, we specify only the queue name.

For a topic, we specify both the topic name and subscription name.

```
Queue:  
CreateReceiver(QueueName)  
  
Topic:  
CreateReceiver(TopicName, SubscriptionName)
```

Q4. Why do we need a subscription name?

Answer:

A topic can contain multiple subscriptions. The subscription name identifies the specific subscription from which the application wants to receive messages.

Q5. What method is used to receive a message?

Answer:

```
await receiver.ReceiveMessageAsync();
```

Q6. What happens to the message when using Peek-Lock?

Answer:

The message is locked when received but is not permanently deleted immediately. After successful processing, we can complete it using:

```
await receiver.CompleteMessageAsync(message);
```

18. Quick Revision

Remember this:

```
QUEUE  
  
Sender  
  ↓  
Queue  
  ↓  
Receiver
```

For a queue:

```
client.CreateReceiver("Q1");
```

TOPIC

Sender



Topic



Subscription



Receiver

For a topic subscription:

```
client.CreateReceiver(  
    "FirstTopic",  
    "S1");
```

Most Important Point

A topic is the publishing destination, but messages are received from subscriptions.

19. Final Interview Summary

Azure Service Bus Topic Message Receiving

1. A publisher sends a message to a **Topic**.
2. The Topic delivers the message to matching **Subscriptions**.
3. A receiver reads the message from a **Subscription**.
4. To create a receiver for a topic subscription, provide:
5. Topic name
6. Subscription name
7. Use `ReceiveMessageAsync()` to receive the message.
8. With Peek-Lock, call `CompleteMessageAsync()` after successful processing.

Final Flow

```
Publisher  
|  
| SendMessageAsync()  
↓
```

